

2026 年度高 2 情報

第2回 デジタルな情報の扱い方

2026/04/18

机に貼ってある番号に従って座ってください
PC の準備は不要です

情報のデジタル化 (p54,p55)

アナログ

連続的に変化する量を連続的な変化を以て表現することを**アナログ**という.

- 温度を水銀柱の長さで測る
- 血圧を手動の血圧計で測る

デジタル

連続的に変化する量を, 時間的にまたは空間的に, 区切りを以て離散的に表現することを**デジタル**という.

- 電子体温計で体温を測る
- デジタル時計で時間を表現する

コンピュータのデジタル表現 (p54,p55)

- デジタルでは全ての情報を0と1の組み合わせで表現できる
 - 電圧の高低やハードディスクでは磁気の向きをそれぞれ0と1に対応させている
- すなわち2進数(2進法)で情報は表現されている
- デジタルデータをアナログデータにすることを**DA変換**
アナログデータをデジタルデータにすることを**AD変換**という
- 変換を行うとデータが失われることがある.
この失われたデータのことを**変換誤差 (AD変換の場合は特に量子化誤差)**という.

情報の単位 (p54,p55)

- 情報量の最小単位をビット (**bit**)
- 8 ビットを 1 バイト (**Byte**) と表現する
 - $1\text{B}=8\text{bit}$
 - $1\text{kB}=1000\text{B}$ ($1\text{KiB}=1024\text{B}$)
 - $1\text{MB}=1000\text{KB}$ ($1\text{MiB}=1024\text{KiB}$)
- 1 秒間に何 bit の情報を伝えられるかを表す単位として **bps** という単位もある. $1\text{kbps}=1000\text{bps}$
- 情報の単位系は少し複雑
 - なぜなら
10 進法を使う人と 2 進法を使う人がいるから
2 進接頭辞が広まらず, 両者のすみわけがなされていないから
 - それによって, 単位系は似て非なるものが 2 種類存在する

SI 接頭辞を用いる方法 (p54,p55)

日本語名	英語名	記号	指数表記
キロ	kilo	k	10^3
メガ	mega	M	10^6
ギガ	giga	G	10^9
テラ	tera	T	10^{12}
ペタ	peta	P	10^{15}
エクサ	exa	E	10^{18}
ゼタ	zetta	Z	10^{21}
ヨタ	yotta	Y	10^{24}

2進接頭辞を用いる方法 (p54,p55)

日本語名	英語名	記号	指数表記
キビ (キロ)	kibi(kilo)	Ki (K)	2^{10}
メビ (メガ)	mebi(mega)	Mi (M)	2^{20}
ギビ (ギガ)	gibi(giga)	Gi (G)	2^{30}
テビ (テラ)	tebi(tera)	Ti (T)	2^{40}
ペビ (ペタ)	pebi(peta)	Pi (P)	2^{50}
エクスビ (エクサ)	exbi(exa)	Ei (E)	2^{60}
ゼビ (ゼタ)	zebi(zetta)	Zi (Z)	2^{70}
ヨビ (ヨタ)	yobi(yotta)	Yi (Y)	2^{80}

情報学では2進接頭辞を使うことが多いが、普及していないので慣習的に KB, MB, GB のような表記が使われることも多い。

2進数の計算 (pp.72-75)

足し算と引き算

$$\begin{array}{r} 1_{(2)} \\ + 1_{(2)} \\ \hline 10_{(2)} \end{array}$$

$$\begin{array}{r} 111_{(2)} \\ + 1_{(2)} \\ \hline 1000_{(2)} \end{array}$$

$$\begin{array}{r} 1_{(2)} \\ - 1_{(2)} \\ \hline 0_{(2)} \end{array}$$

$$\begin{array}{r} 100_{(2)} \\ - 1_{(2)} \\ \hline 11_{(2)} \end{array}$$

- 多くのコンピュータは足し算や引き算の片方しかできない (減算回路と加算回路両方をのせるのは非効率)
- よって、足し算のみで引き算を、または引き算のみで足し算を表す必要がでてくる

補数について (pp.72-75)

2の補数 (高校範囲での「補数」)

- 補数とはある数に対して、足すと1桁増えるもっとも小さな数のこと
- $011_{(2)}$ の(2進数での)補数は $101_{(2)}$
実際、2つを足すと $1000_{(2)}$ となり、位が上がる.

(参考)1の補数

- 各ビットを0と1を反転させた数のこと
- 1を足すと2の補数になる

補数を使うと2進数の引き算が足し算を以てできるようになる

補数の利用 (pp.72-75)

補数を使った引き算

Q $0110_{(2)} - 0101_{(2)}$ を計算せよ

A $0101_{(2)}$ の (2 の) 補数は $1011_{(2)}$ であるから,

$$\begin{array}{r} 0110_{(2)} \\ + 1011_{(2)} \\ \hline 1\boxed{0001}_{(2)} \end{array}$$

よって、答えは $0001_{(2)}$

4bit の計算だから、4bit 分だけしか機械は確認できない。

このように補数は負の数の側面をもつ。

コンピュータでの数の扱い (pp.72-75)

整数

- 符号なし整数は n 個の数字列で, 0 から $2^n - 1$ までの数字を表すことができる.

3bit であれば, $000_{(2)}$ から $111_{(2)}$ まで

- 符号付き整数は n 個の数字列で, -2^{n-1} から $2^{n-1} - 1$ までの数を表すことができる.

3bit であれば, $100_{(2)} = -4_{(10)}$ から $011_{(2)} = 3_{(10)}$ まで

負の数 (最上位 bit が 1 の数) は 2 の補数を取ることで符号を反転した符号なし整数になる.

$101_{(2)} = -3_{(10)} \rightarrow (2 \text{ の補数}) \rightarrow 011_{(2)} = 3_{(10)}$

コンピュータでの数の扱い (pp.72-75)

2進数での小数

- 例えば, $0.1_{(2)} = \frac{1}{2_{(10)}} = 0.5_{(10)}$, $0.11_{(2)} = \frac{1}{2_{(10)}} + \frac{1}{4_{(10)}} = 0.75_{(10)}$
- 小数第 n 位は2進数であれば, 2^{-n} を表している

16進数の場合

- m 進数 ($m \geq 11$) は基本的に数が足りない
- $10_{(10)} \rightarrow A_{(m)}, \dots, 15_{(10)} \rightarrow F_{(m)}, \dots$ のように不足分はアルファベットを使う
- 情報分野では2進数と16進数の変換をすることが多い

コンピュータでの実数の表現 (pp.72-75)

浮動小数点数 (floating point number)

31	30	29	28	...	24	23	22	21	...	1	0
S	E 8bit						M 23bit				

- Sを符号部という
- Eを指数部という
- Mを仮数部という
- $(-1)^S \times 1.M \times 2^{E-127}$ という数をこれによって表す

浮動小数点数の例 (p74)

例題 4

- 1 10000010 111001000000000000000000 という 32bit の数を考える
- まず負の数 (最上位 bit が 1)
- 指数部: $10000010_{(2)} = 130$
- 仮数部: $111001000000000000000000_{(2)}$
→ $1.111001_{(2)} = 1 + 2^{-1} + 2^{-2} + 2^{-3} + 2^{-6}$
- 以上より, この浮動小数点数が表す値 (10 進数) は,
 $(-1) \times 2^{130-127} \times (1 + 2^{-1} + 2^{-2} + 2^{-3} + 2^{-6}) =$
 $-(8 + 4 + 2 + 1 + 0.125) = -15.125$

コンピュータの演算の誤差 (p75)

オーバーフロー

- 4bit で符号なし整数の演算を試みる
 $15_{(10)} + 2_{(10)} = 1111_{(2)} + 0010_{(2)} = 1\boxed{0001}_{(2)} = 1$
- 4bit で符号付き整数の演算を試みる
 $6_{(10)} + 2_{(10)} = 0110_{(2)} + 0010_{(2)} = 1000_{(2)} = -8_{(10)}$
- コンピュータは演算できる数の範囲が決まっている. それを超えると**オーバーフロー**となり, 誤差が生じる

コンピュータの演算の誤差 (p75)

アンダーフロー

- コンピュータは絶対値が小さすぎる数を扱えない
- 32bit の浮動小数点数でももちろん下限はある
- それより小さい数は0と処理されてしまうことがある. これを**アンダーフロー**という.

コンピュータの演算の誤差 (p75)

丸め誤差

- 特に小数を含む数を有限 bit で表現する際に生じる誤差を**丸め誤差**という. (浮動小数点数などでも当然生じる)
- $0.6_{(10)} = 2^{-1} + 2^{-4} + 2^{-5} + 2^{-8} + \dots = 0.100110011001\dots_{(2)} = 0.\dot{1}00\dot{1}$ これを小数第8位までで表現すると $0.10011001_{(2)}$ となる.
- $0.10011001_{(2)} = 0.59765625_{(10)}$

コンピュータの文字の扱い方 (p56)

- デジタルの世界では、数字を使って文字を表している。
この取り決めを**文字コード**という。
- 1バイトで $2^8 = 256$ 種類. 英数字などは1バイトあればよい。
- 2バイトで $2^{16} = 65536$ 種類. 漢字などは2バイト以上必要。
- 日本で使われている文字コードには、**JIS コード**, **シフト JIS コード**, **EUC** などがある
- 端末ごとに採用している文字コードが異なるので、同じコードでも別の文字に対応していること (**文字化け**) やそもそも対応していない文字 (**機種依存文字**) が存在することがある。
- 様々な文字を扱える文字コードとしては **Unicode** が有名。
- Unicode の1つである **UTF-8** は可変長バイト (1-4 バイト) で文字を表す (注：教科書 p56 最終行は誤植と思われる)。

AD 変換の手法 (p57)

基本的に物理量は、音や光であれば「波」、音の大きさや光の明るさなどは波の「振幅」などのアナログな情報をどうにかしてデジタルにする手法を考えたい。

標本化

- アナログの情報を一定の間隔で区切り、区間ごとのアナログ量を取り出す操作を**標本化**という
- この間隔のことを**標本化周期 (サンプリング周期)** という
- 取り出した点を**標本点**という

AD 変換の手法 (p57)

量子化

- 各標本点のアナログ量を離散な値 (整数値など不連続な値) に変換する操作を**量子化**という.

注 教科書には最も近い段階値に揃えるとあるが, 必ずしも最も近い段階値に揃えるとは限らない.

- 量子化の段階数を決めるものを**量子化 bit 数**といい, 量子化 bit 数が n のとき, 2^n 段階となる.
- アナログ量と段階値のずれを**量子化誤差**という
- 量子化 bit 数が多ければ多いほど量子化誤差は小さくなり, 各段階とのずれは小さくなる (正確にデジタルに反映される)
- 量子化した値を2進数で表現することを**符号化**という

AD 変換の手法 (p57)

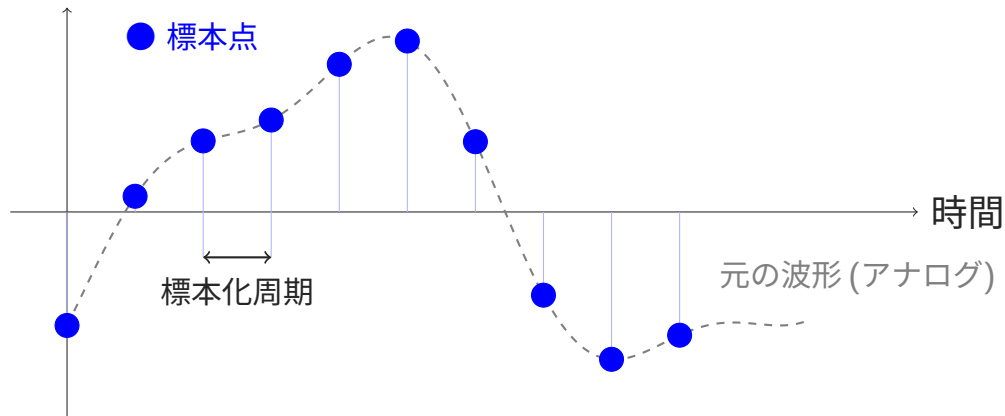
標本化定理

- 標本化周期が長いほど, 元の波形の再現性が低くなる
- 逆に短いほど, 元の波形をより細かく再現できる
一方で, データ量は大きくなる
- 「元のデータの周期の半分より短い標本化周期で標本化すると理論上元の波が再現できる」というのが**標本化定理**の主張である
- 証明は大学レベル数学 (フーリエ変換/逆フーリエ変換) を要するのでここでは取り扱わない。

AD変換の手法 (p57)

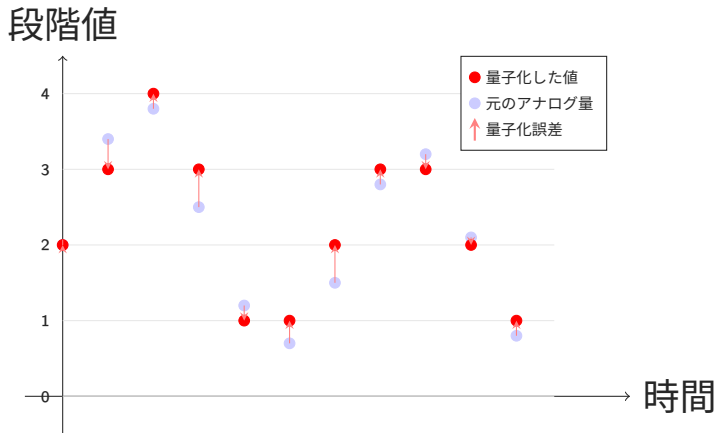
標本化の図

アナログ量



AD変換の手法 (p57)

量子化 (今回は一番近い段階値に揃えている) の図



符号化はこの段階値を2進数で表現することである.

今週の課題

- 1 2進数と16進数および2進数と10進数の間の計算をする
 - 2 2進数の小数を計算できるようになる
 - 3 浮動小数点数を理解する
- 2進数と16進数の変換は下位 bit から順に4桁区切りで直せばよいので難しくはない.

$$10\ 0101\ 1001\ 1101_{(2)} \leftrightarrow 259D_{(16)}$$

- 毎週の累計としておおむね5点を加点予定
- 白紙は提出なしと同じものとする. わからなくても計算の痕跡などを残すこと. 出来の良しあしはそこまで関係ない.
- 来週以降, 授業内において, PCでClassroomを使用することがある. PW等の記録を思い出しておくこと.